ENTERPRISE SOLUTION ARCHITECTURE

CAS Education ERP & CRM Platform

Technical Design & Specification Report

Audience: Backend, Frontend, Database, DevOps & QA Engineers
Scope: Architecture · Technology Stack · Modules · Database · API · Security
Version: 1.0 **Date:** July 2026
Status: Issued for Development
Companion: schema.sql (MySQL 8 DDL, 50 tables)

CAS Education ERP & CRM Platform

Technical Design & Specification Report (for Development Team)

Document	Technical Report — Architecture, Modules, Database & Technology
Project	CAS Education ERP & CRM Platform
Audience	Backend, Frontend, Database, DevOps & QA engineers
Version	1.0
Date	July 2026
Status	Issued for development
Companion file	`schema.sql` (MySQL 8 DDL)

> **Purpose.** This report is the single technical source of truth a senior team can build from without re-asking business questions. It defines the architecture, the technology stack and why, each module's responsibilities and data, the complete MySQL data model, the API and security model, and the cross-cutting concerns (caching, queues, notifications, storage, logging, deployment). Business rationale lives in the BRD; this document is implementation-facing.

1. System Overview

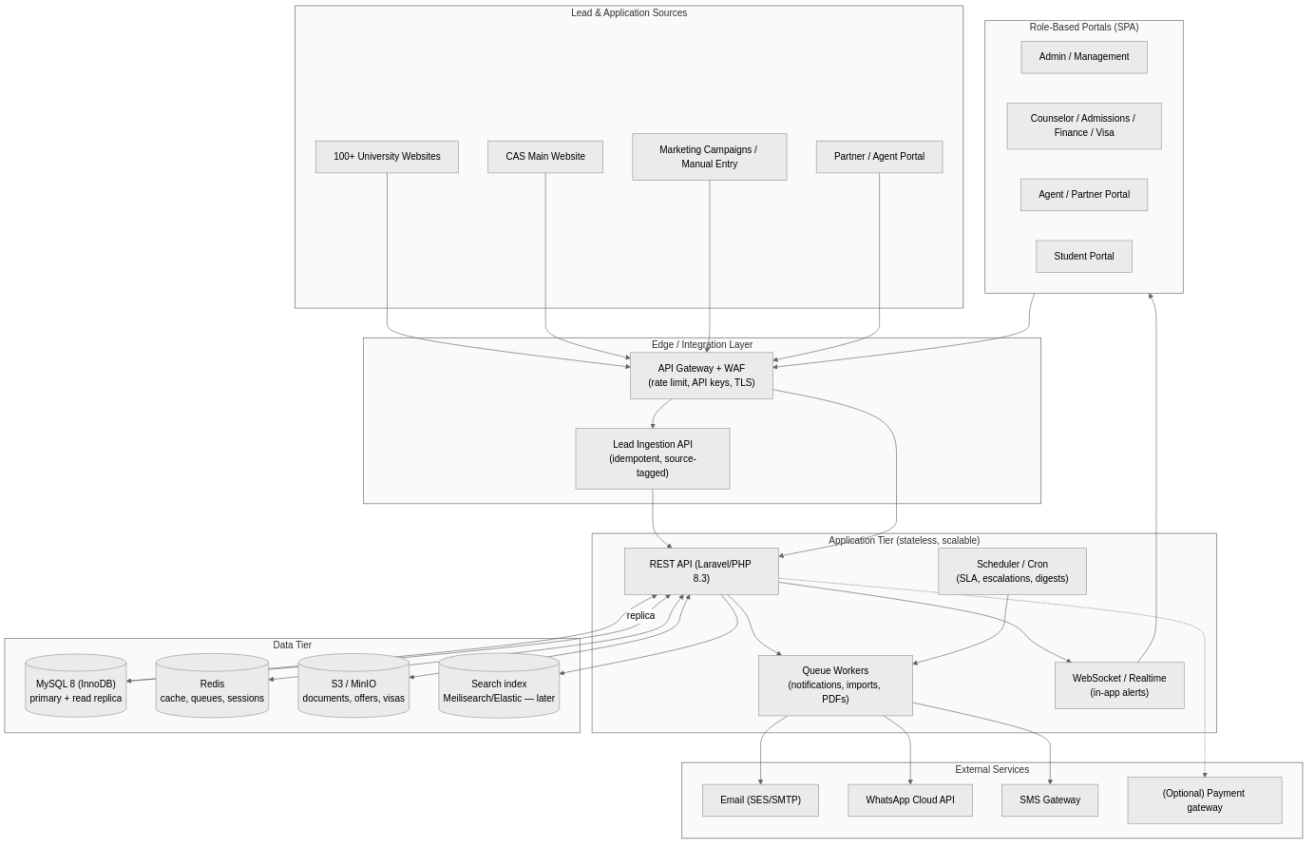
CAS is an international education consultancy placing students into universities in China. Leads flow in from **100+ university websites**, agents, campaigns and staff, and are driven through a controlled departmental lifecycle — **Lead → Admin assignment → Counselor qualification → Admissions → Finance → Visa → Pre-arrival → Arrival & first-year payment** — while preserving **one permanent student identity** and a **full audit trail**. The commercial core is a **commission lifecycle** that separates forecast revenue from cash actually received, then distributes shares down an agent → sub-agent → sub-sub-agent hierarchy.

1.1 Architectural drivers

Driver	Implication for design
100+ websites as lead sources	**API-first** ingestion, idempotent, source-attributed, rate-limited, per-source API keys
Single student identity	Central DB with **duplicate detection** on intake (email/phone/passport fuzzy match)
Multi-department workflow	**State-machine driven** records with status history + assignment + SLA/escalation
Commission accuracy	Explicit **commission state model** and immutable financial history
Least-privilege security	**RBAC** with record-scope (own / assigned / department / branch / global)
Accountability & compliance	**Immutable audit log**, soft deletes, document access logging
Growth (universities, countries, branches, agents)	Multi-tenant-friendly, horizontally scalable stateless API + Redis + queues

2. High-Level Architecture

The platform is a **centralized API-first system**: all public websites and role-based portals connect to one backend and one database. This avoids per-website admin panels and databases while preserving each inquiry's source.



2.1 Logical layers

1. **Presentation** — SPA portals (React + TypeScript) and public website widgets/forms.
2. **API / Gateway** — REST, versioned (`/api/v1`), authenticated (Sanctum/JWT), rate-limited, WAF-fronted.
3. **Application / Domain** — controllers → services → domain models; workflow engine; policy-based authorization.
4. **Data & Integration** — MySQL (source of truth), Redis (cache/queue/session), object storage (documents), external messaging providers.
5. **Async** — queue workers + scheduler for notifications, imports, PDF generation, SLA checks, escalations.

---

3. Technology Stack

MySQL is mandated. The remainder is a **recommended, defensible stack** optimized for a data-dense ERP/CRM with heavy RBAC, workflow, queues and messaging. Alternatives are noted where a team may reasonably differ.

Layer	Recommended	Why	Reasonable alternative
Language/Runtime	PHP 8.3	Mature ERP ecosystem, fast delivery, first-class MySQL	Node.js 20 (TS)
Backend framework	**Laravel 11**	Built-in queues (Horizon), mail, scheduler, policies, Sanctum auth, Eloquent ORM, migrations; huge ERP/CRM fit	NestJS (Node) / Django (Python)
RBAC	`spatie/laravel-permission`	Roles, permissions, guard integration, well-tested	Custom policy layer
Frontend	**React 18 + TypeScript**	Component ecosystem for data-dense admin UIs	Vue 3 + TS
Admin UI kit	Ant Design **or** MUI	Tables, forms, filters, date pickers out of the box	Mantine / shadcn/ui
Client data/state	TanStack Query + Zustand	Server-cache + light client state	Redux Toolkit
Build/host (FE)	Vite; served via Nginx/CDN	Fast builds, SPA hosting	Next.js (if SSR needed)
Database	**MySQL 8 (InnoDB)**	Mandated; transactions, FKs, JSON columns, full-text	—
Cache/Sessions/Queue broker	**Redis 7**	Cache, queues, rate-limit buckets, sessions	—
Queue processing	Laravel Horizon (Redis)	Dashboarded async workers	BullMQ (Node)
Object storage	S3-compatible (AWS S3 / MinIO)	Secure, scalable document store, pre-signed URLs	Azure Blob / GCS
Realtime	Laravel Reverb / Pusher (WebSockets)	In-app notifications, live pipeline	Socket.io
Search (phase 2)	Meilisearch → Elasticsearch	Fast student/lead search at scale	MySQL full-text (initial)
Email	Amazon SES / SMTP	Transactional + templated	Postmark
WhatsApp	Meta WhatsApp Cloud API	Official templated messaging	Twilio WhatsApp
SMS	Twilio / local gateway	OTP, alerts	Vonage
PDF generation	`dompdf` / `wkhtmltopdf` / Gotenberg	Invoices, offers, statements	Puppeteer service
Auth	Laravel Sanctum (SPA token) + optional 2FA	SPA + API tokens; agent/student portals	JWT (tymon)
Containerization	Docker + Docker Compose	Reproducible envs	—
Reverse proxy	Nginx	TLS termination, static, gzip	Traefik
CI/CD	GitHub Actions	Test → build → deploy	GitLab CI

Layer	Recommended	Why	Reasonable alternative
Monitoring	Sentry (errors) + Prometheus/Grafana	Errors + metrics	Datadog
Log aggregation	Loki/Grafana or ELK	Centralized structured logs	CloudWatch
IaC (optional)	Terraform	Reproducible infra	Ansible

3.1 Environment matrix

Environment	Purpose	Data	Notes
Local	Developer machines	Seeded/fake	Docker Compose parity
Dev	Integration	Synthetic	Auto-deploy on merge to `develop`
Staging	UAT / client sign-off	Anonymized prod-like	Mirrors prod config
Production	Live	Real	Blue-green or rolling deploy

---

4. Cross-Cutting Technical Concerns

4.1 Authentication & Authorization

- **AuthN:** Token-based (Sanctum). Separate guards for **staff**, **agents**, **students**. Optional TOTP 2FA for staff/finance. Password policy, lockout, and session/token expiry configurable.
- **AuthZ:** RBAC via roles → permissions, plus **record-scope policies** enforcing `own / assigned / department / branch / global`. Every API action passes through a Policy (see §8 and Phase 7 matrix). Deny-by-default.
- **API keys:** Per-website/source keys for lead ingestion, individually revocable and rate-limited.

4.2 Data integrity & soft delete

- All business tables carry `created_at`, `updated_at`, `created_by`, `updated_by`, and `deleted_at` (soft delete). Hard deletes are prohibited on business-critical tables.
- Financial and commission records are **append-only where possible**; corrections create adjusting entries, not overwrites.

4.3 Auditing

- Global audit interceptor writes to `audit_logs` on create/update/delete for critical entities (student, application, invoice, payment, commission, visa_case, user, role). Captures actor, action, entity, before/after JSON, IP, user-agent, timestamp. Immutable (no update/delete).
- Sensitive **document access** is logged to `document_access_logs`.

4.4 Caching strategy

Cache	What	TTL / invalidation
Config/catalog	Universities, courses, scholarships, settings	Long TTL; bust on write
Dashboards	Aggregates (funnel, revenue)	5–15 min TTL; scheduled refresh
Permissions	Role→permission maps	Bust on role change
Rate limits	Per-IP / per-API-key buckets	Sliding window (Redis)

4.5 Queues & scheduling

- **Queues (Redis/Horizon):** notifications (email/WhatsApp/SMS), website lead imports, PDF generation, bulk assignment, report exports, webhook retries.
- **Scheduler (cron):** SLA breach checks → escalations, overdue-task reminders, commission-eligibility sweeps, daily/weekly management digests, receivables aging refresh.

4.6 Notifications

- Channels: **In-app (WebSocket)**, **Email**, **WhatsApp**, **SMS**. Unified `notifications` table + per-channel delivery logs. Template-driven with variables. Preferences per user/role. Escalation ladders (owner → manager → head).

4.7 File storage & security

- Documents stored in S3/MinIO under keyed paths (`students/{id}/...`), never public. Access via **short-lived pre-signed URLs** brokered by the API after a policy check. Virus scanning hook on upload. Allowed types/size enforced server-side.

4.8 Observability

- Structured JSON logs with correlation IDs; errors to Sentry; metrics (request latency, queue depth, job failures, DB slow queries) to Prometheus/Grafana; uptime/health endpoints (`/health`, `/ready`).

4.9 Non-functional targets (summary)

Concern	Target
API p95 latency	< 400 ms for standard reads
Availability	99.5%+ (prod)
Lead ingestion	Idempotent, ≥ 50 req/s per source burst-tolerant
Backups	Daily full + hourly incremental (binlog); PITR
RPO / RTO	RPO ≤ 1h; RTO ≤ 4h
Concurrent users	Thousands (stateless API scales horizontally)
Security	Least privilege, encryption in transit (TLS) & at rest, audit logging

---

5. Module Breakdown (Technical)

Each module lists its **responsibility**, **primary entities**, **key statuses/state machine**, **core operations/endpoints**, and **notable business logic**. Entity names map 1:1 to the tables in §6 and `schema.sql`.

5.1 Identity, Org & Access (Foundation)

- **Responsibility:** Branches, departments, countries, users, roles, permissions, settings, API keys.
- **Entities:** `branches`, `departments`, `countries`, `users`, `roles`, `permissions`, `role_permissions`, `model_has_roles`, `settings`, `api_sources`, `api_keys`.
- **Core ops:** CRUD users; assign roles; configure branch/department scoping; issue/revoke source API keys; manage global settings (SLAs, statuses, templates).
- **Logic:** Deny-by-default RBAC; record scope resolved per request; every write audited.

5.2 Lead & Student CRM

- **Responsibility:** Capture leads (both intake routes), duplicate detection, source/campaign/website attribution, assignment, notes, follow-ups, full student timeline.
- **Entities:** `leads`, `students`, `student_guardians`, `activities` (timeline/notes), `tasks`, `marketing_sources`, `campaigns`, `website_sources`.
- **Lead statuses:** `new → assigned → contacted → qualified → converted` | `unqualified` | `lost` | `duplicate`.
- **Student statuses:** `active → in_process → enrolled → arrived` | `withdrawn` | `inactive`.
- **Core ops:** ingest lead (API), dedup-check, assign (rule/manual), qualify, convert lead→student(+application), log activity, create task, reassign.
- **Logic — duplicate detection:** on intake, match on normalized email, phone (E.164), and passport number; fuzzy name+DOB. Exact match → link to existing student and flag as `duplicate` lead; probable match → route to Admin review queue.

- **Logic — assignment:** configurable rules (round-robin, by branch, by source, by workload). Auto-generates a task for the assignee with SLA due date.

5.3 University, Course & Scholarship Catalog

- **Responsibility:** University profiles, programs/courses, fees, requirements, intakes, deadlines, scholarships, assigned staff, and **commission agreements**.
- **Entities:** `universities`, `university_contacts`, `courses`, `intakes`, `scholarships`, `university_agreements`.
- **Core ops:** CRUD catalog; define intake windows & deadlines; define scholarship criteria; define commission terms per university (amount/%/tiered, currency, eligibility trigger).
- **Logic:** `university_agreements` drives **expected commission** computation for every application to that university/course/intake.

5.4 Admissions Management

- **Responsibility:** Eligibility review, document checklist, university/scholarship submission, references, offer letters, outcomes. Supports **multiple applications per student**.
- **Entities:** `applications`, `application_status_history`, `documents` (linked), `offers`, `application_reviews`.
- **Application statuses:** `draft → under_review → documents_pending → submitted → offer_received → accepted → rejected → withdrawn → deferred`.
- **Core ops:** create application (student × university × course × intake), attach checklist docs, submit to university, record offer, accept/reject, transfer to finance/visa.
- **Logic:** offer acceptance triggers **invoice creation** (finance) and sets commission to `expected`; document checklist completeness gates submission.

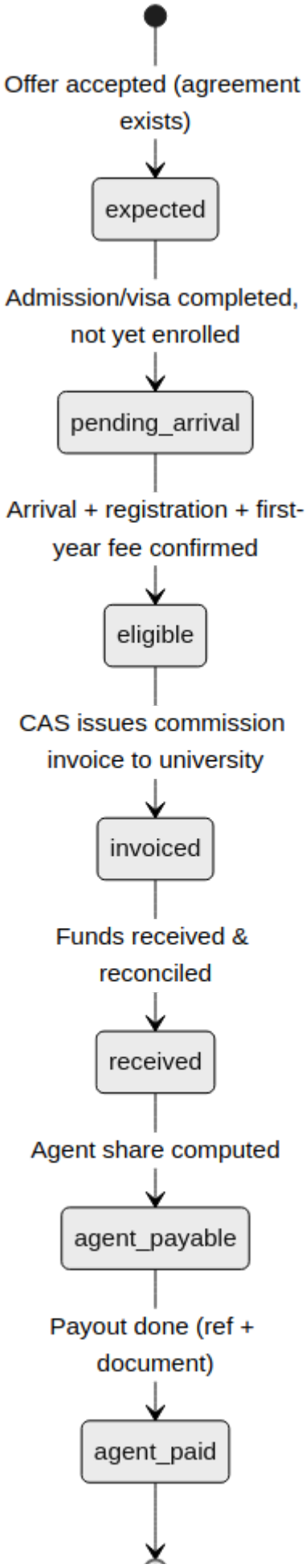
5.5 Documents

- **Responsibility:** Secure, structured storage linked to student/application/visa/finance; checklist tracking; access logging.
- **Entities:** `documents`, `document_types`, `document_access_logs`.
- **Core ops:** upload (pre-signed), categorize, verify/approve, link to entity, download (policy-checked, logged), version.
- **Logic:** polymorphic link (`documentable_type`, `documentable_id`); status `pending → verified → rejected`; sensitive types (passport, financials) restricted.

5.6 Finance, Invoicing & Commission

- **Responsibility:** Student/agent/university invoices, receipts, partial payments, expected commission, university receivables, agent payables — separating forecast from cash.
- **Entities:** `invoices`, `invoice_items`, `payments`, `commissions`, `commission_status_history`, `agent_payables`, `agent_payments`.
- **Invoice statuses:** `draft → issued → partially_paid → paid → cancelled → overdue`.

- **Commission lifecycle (from proposal):** `expected → pending_arrival → eligible → invoiced → received`; agent side `agent_payable → agent_paid`.
- **Core ops:** issue invoice, record payment/receipt, advance commission state, compute agent payable from share rules, record agent payout (with reference + document).
- **Logic — commission timing:** CAS is eligible for university commission **only after student arrives + registers + pays first-year tuition**. State transitions are gated by verified milestone events; each transition is written to `commission_status_history` (immutable). Agent payable is computed from `agent_agreements` share structure across the hierarchy.


Offer accepted (agreement exists)
expected
Admission/visa completed, not yet enrolled
pending_arrival
Arrival + registration + first-year fee confirmed
eligible
CAS issues commission invoice to university
invoiced
Funds received & reconciled
received
Agent share computed
agent_payable
Payout done (ref + document)
agent_paid

5.7 Agent / Partner Management & Portal

- **Responsibility:** Partner, sub-agent and sub-sub-agent hierarchy; multi-student submission; application/commission visibility; statements; service charges; agreements.
- **Entities:** `agents` (self-referencing hierarchy via `parent_agent_id`), `agent_agreements`, `agent_payables`, `agent_payments`; links via `students.agent_id` / `applications.agent_id`.
- **Core ops:** onboard agent, define share agreement, submit students, track applications, view commission visibility (own records only), generate statements.
- **Logic:** commission distribution cascades University → CAS → Partner → Sub-agent → Sub-sub-agent per configurable share; agent portal is **restricted to own records** (RBAC scope = own).

5.8 Visa Management

- **Responsibility:** Dedicated visa case, document checklist, appointment, embassy submission, decision, travel clearance.
- **Entities:** `visa_cases`, `visa_checklist_items`, `visa_status_history`, linked `documents`.
- **Visa statuses:** `not_started → documents_collection → appointment_booked → submitted → approved → rejected → travel_ready`.
- **Core ops:** open visa case (post-admission), manage checklist, book appointment, record submission/decision, mark travel-ready → notify counselor for pre-arrival.
- **Logic:** approval + travel-ready feeds the arrival milestone that (with finance confirmation of first-year payment) drives commission `eligible`.

5.9 Task, Workflow & Communication

- **Responsibility:** Workflow-driven handovers, tasks, SLAs, escalations, and unified communication (email/WhatsApp/SMS/in-app).
- **Entities:** `tasks`, `workflow_definitions`, `workflow_transitions`, `automation_rules`, `notifications`, `communications`, `message_logs`.
- **Core ops:** auto-create next-owner task on transition, SLA timers, escalation ladders, templated messaging, communication log against student.
- **Logic:** each department transition = a workflow transition with owner + due date; overdue → escalate per ladder; all outbound comms logged to the student timeline.

5.10 Management Analytics & Reporting

- **Responsibility:** Lead funnel, conversion, admissions, scholarship success, counselor/agent/source/university performance, finance forecast, receivables, commission pipeline.
- **Entities:** read models / SQL views over the transactional tables (`v_lead_funnel`, `v_commission_pipeline`, `v_agent_performance`, etc.).

- **Core ops:** dashboard aggregates (cached), drill-down, scheduled exports (PDF/Excel), role-scoped visibility.
- **Logic:** aggregates respect RBAC scope; heavy queries served from read replica + Redis cache; nightly materialization for expensive reports.

5.11 Student Portal

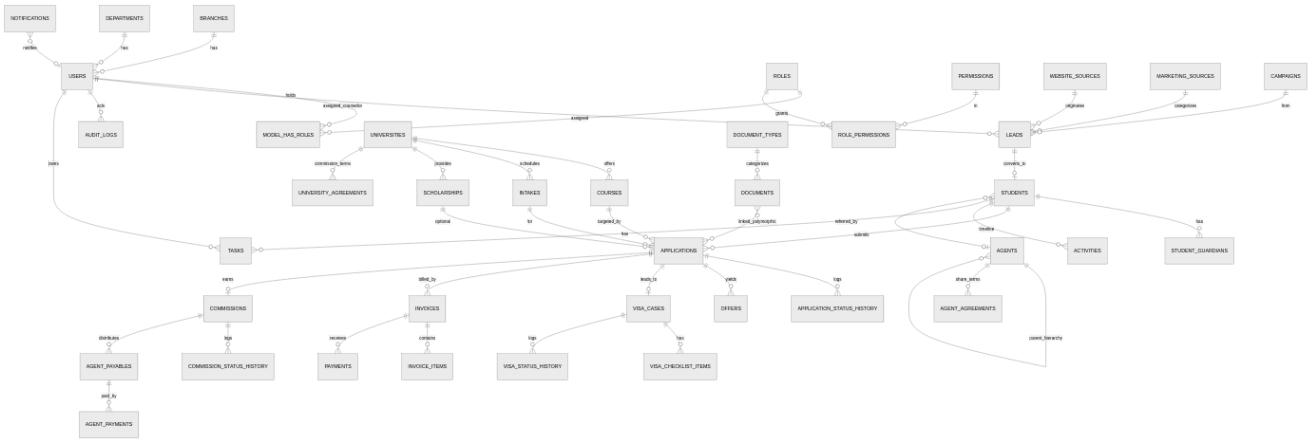
- **Responsibility:** Application progress, tasks, document upload, messages, invoices/receipts, visa status, downloadable letters.
- **Entities:** read/limited-write access to `applications`, `documents`, `invoices`, `payments`, `visa_cases`, `communications` — **own records only**.
- **Logic:** separate `student` auth guard; strictly scoped; upload allowed, financial/commission internals hidden.

---

6. Database Design (MySQL 8)

Conventions: InnoDB, `utf8mb4`; `BIGINT UNSIGNED` PKs (`id`); snake_case; every business table has `created_at`, `updated_at`, `created_by`, `updated_by`, `deleted_at` (soft delete). Money as `DECIMAL(15,2)` + `currency CHAR(3)`. Enums implemented as `VARCHAR` + application constants (portable) or `ENUM` where fixed. FKs `RESTRICT` on delete for master data, `CASCADE` only for pure child rows. Full DDL is in `schema.sql`.

6.1 Entity-Relationship Diagram (core)



6.2 Table catalogue (key columns)

The following summarizes the principal tables. Datatypes, nullability, defaults, keys and indexes are fully expressed in `schema.sql`.

Foundation

Table	Key columns	Notes
`branches`	id, name, code(UQ), country_id(FK), city, is_active	Org unit
`departments`	id, name, code(UQ), is_active	Counseling/Admissions/Finance/Visa/Marketing
`countries`	id, name, iso2(UQ), phone_code	Reference
`users`	id, name, email(UQ), phone, password, branch_id(FK), department_id(FK), status, last_login_at, 2fa_secret	Staff
`roles`	id, name(UQ), guard_name	spatie-compatible
`permissions`	id, name(UQ), guard_name	e.g. `leads.view.assigned`
`role_permissions`	role_id(FK), permission_id(FK)	composite PK
`model_has_roles`	role_id(FK), model_type, model_id	polymorphic user↔role
`settings`	id, key(UQ), value(JSON), group	SLAs, templates, toggles
`api_sources`	id, name, type(website/agent/campaign), website_url, is_active	the 100+ sources
`api_keys`	id, api_source_id(FK), key_hash(UQ), scopes, rate_limit, revoked_at	per-source ingestion keys

CRM

Table	Key columns	Notes
`marketing_sources`	id, name, channel, is_active	organic/paid/referral etc.
`campaigns`	id, name, marketing_source_id(FK), utm_*, start/end	attribution
`website_sources`	id, api_source_id(FK), domain(UQ), university_focus	maps a site to a source
`leads`	id, first_name, last_name, email, phone, passport_no, country_id(FK), website_source_id(FK), marketing_source_id(FK), campaign_id(FK), agent_id(FK), assigned_user_id(FK), status, intake_route(info/full), student_id(FK), duplicate_of(FK), created_at	intake record
`students`	id, student_code(UQ), first_name, last_name, email, phone, passport_no, dob, gender, country_id(FK), branch_id(FK), agent_id(FK), assigned_counselor_id(FK), status	permanent identity
`student_guardians`	id, student_id(FK), name, relation, phone, email	optional
`activities`	id, student_id(FK), user_id(FK), type(note/call/email/status), body, meta(JSON), created_at	timeline
`tasks`	id, title, description, assignee_id(FK), student_id(FK), related_type, related_id, due_at, priority, status, escalated_at	workflow tasks

Catalog

Table	Key columns	Notes
`universities`	id, name, code(UQ), country_id(FK), city, website, status	profile
`university_contacts`	id, university_id(FK), name, email, phone, role	comms
`courses`	id, university_id(FK), name, level(bachelor/master/phd), duration_months, tuition_fee, currency, requirements(JSON), status	programs
`intakes`	id, university_id(FK), course_id(FK), name, start_date, application_deadline, seats, status	windows
`scholarships`	id, university_id(FK), name, type, coverage, criteria(JSON), deadline, status	scholarships
`university_agreements`	id, university_id(FK), course_id(FK, null=all), commission_type(fixed/percent/tiered), commission_value, currency, eligibility_trigger, valid_from, valid_to	drives expected commission

Admissions & Documents

Table	Key columns	Notes
`applications`	id, application_code(UQ), student_id(FK), university_id(FK), course_id(FK), intake_id(FK), scholarship_id(FK,null), agent_id(FK), assigned_admissions_id(FK), status, submitted_at	multiple per student
`application_status_history`	id, application_id(FK), from_status, to_status, changed_by(FK), note, created_at	immutable
`offers`	id, application_id(FK), offer_type(conditional/unconditional), reference_no, issued_date, expiry_date, document_id(FK), status	
`application_reviews`	id, application_id(FK), reviewer_id(FK), eligibility_result, checklist_complete, notes	
`document_types`	id, name(UQ), category, is_sensitive, required_for(JSON)	passport, transcript...
`documents`	id, document_type_id(FK), documentable_type, documentable_id, file_path, file_name, mime, size, status(pending/verified/rejected), verified_by(FK), version	polymorphic
`document_access_logs`	id, document_id(FK), user_id(FK), action(view/download), ip, created_at	compliance

Finance & Commission

Table	Key columns	Notes
`invoices`	id, invoice_no(UQ), party_type(student/agent/university), party_id, application_id(FK,null), currency, subtotal, tax, total, amount_paid, status, issued_at, due_at	multi-party
`invoice_items`	id, invoice_id(FK), description, qty, unit_price, amount	lines
`payments`	id, payment_no(UQ), invoice_id(FK), amount, currency, method, reference, received_at, recorded_by(FK), document_id(FK)	receipts
`commissions`	id, application_id(FK,UQ), university_id(FK), agreement_id(FK), currency, expected_amount, eligible_amount, invoiced_amount, received_amount, status, eligibility_confirmed_at	lifecycle
`commission_status_history`	id, commission_id(FK), from_status, to_status, amount, changed_by(FK), reference, created_at	immutable
`agents`	id, agent_code(UQ), name, type(partner/sub/sub_sub), parent_agent_id(FK self), email, phone, country_id(FK), status	hierarchy
`agent_agreements`	id, agent_id(FK), share_type(percent/fixed), share_value, applies_to(JSON), valid_from, valid_to	distribution rules
`agent_payables`	id, commission_id(FK), agent_id(FK), amount, currency, status(payable/paid), computed_at	derived
`agent_payments`	id, agent_payable_id(FK), amount, method, reference, document_id(FK), paid_at, recorded_by(FK)	payout evidence

Visa

Table	Key columns	Notes
`visa_cases`	id, application_id(FK), student_id(FK), assigned_visa_officer_id(FK), status, appointment_at, submitted_at, decision_at, travel_ready_at	dedicated case
`visa_checklist_items`	id, visa_case_id(FK), name, is_required, document_id(FK,null), status	checklist
`visa_status_history`	id, visa_case_id(FK), from_status, to_status, changed_by(FK), note, created_at	immutable

Communication, Workflow & System

Table	Key columns	Notes
`workflow_definitions`	id, name, entity_type, states(JSON), transitions(JSON), sla(JSON)	configurable
`workflow_transitions`	id, entity_type, entity_id, from_state, to_state, actor_id(FK), created_at	audit of moves
`automation_rules`	id, name, trigger(JSON), conditions(JSON), actions(JSON), is_active	no-code automation
`notifications`	id, user_id(FK), type, title, body, channels(JSON), read_at, created_at	in-app + fanout
`communications`	id, student_id(FK), channel(email/whatsapp/sms), direction, template, body, status, sent_by(FK), created_at	logged comms
`message_logs`	id, communication_id(FK), provider, provider_msg_id, status, error, created_at	delivery tracking
`audit_logs`	id, user_id(FK), action, auditable_type, auditable_id, old_values(JSON), new_values(JSON), ip, user_agent, created_at	immutable

6.3 Indexing strategy (highlights)

- Unique: `students.student_code`, `students.passport_no` (nullable-unique), `applications.application_code`, `invoices.invoice_no`, `commissions.application_id`.
- Search/filters: composite indexes on `leads(status, assigned_user_id, created_at)`, `applications(status, assigned_admissions_id)`, `commissions(status, university_id)`, `tasks(assignee_id, status, due_at)`.
- Dedup: indexes on normalized `leads.email`, `leads.phone`, `leads.passport_no`.
- Polymorphic: `documents(documentable_type, documentable_id)`.
- All FKs indexed. Read-heavy reports served from replica + covering indexes/views.

6.4 Views (for analytics)

`v_lead_funnel`, `v_conversion_by_counselor`, `v_commission_pipeline`, `v_receivables_aging`, `v_agent_performance`, `v_university_performance`, `v_application_funnel` — thin SQL views over transactional tables, RBAC-scoped in the application layer.

6.5 Soft-delete & history strategy

- Soft delete via `deleted_at`; global query scope excludes deleted rows.
- Status/lifecycle changes are recorded in dedicated **immutable history tables** (`*_status_history`) rather than mutating the row's audit trail.
- Financial corrections are adjusting entries; original records are never overwritten.

---

7. API Design

RESTful, versioned under `/api/v1`, JSON in/out, token-authenticated (Sanctum), rate-limited, policy-authorized. Standard envelope, pagination, filtering and sorting apply to all list endpoints.

7.1 Conventions

- **Auth:** `Authorization: Bearer <token>`; guards: `staff`, `agent`, `student`.
- **Versioning:** URL prefix `/api/v1`.
- **Pagination:** `?page=1&per_page=25` → response `meta{ current_page, per_page, total, last_page }`.
- **Filtering:** `?filter[status]=submitted&filter[branch_id]=3`.
- **Sorting:** `?sort=-created_at,last_name`.
- **Search:** `?q=term` (indexed fields).
- **Idempotency (ingestion):** `Idempotency-Key` header on lead ingestion to prevent duplicates on retry.
- **Errors:** consistent shape `{ "message": "...", "errors": { "field": ["..."] }, "code": "VALIDATION_ERROR" }`.
- **HTTP codes:** 200/201 success, 401 unauth, 403 forbidden (policy), 404 not found, 409 conflict (dedup), 422 validation, 429 rate limit.

7.2 Representative endpoints

Domain	Method & path	Purpose	Auth scope
Ingestion	`POST /api/v1/ingest/leads`	Website/agent lead intake (idempotent, API key)	source key
Leads	`GET/POST /api/v1/leads` · `PATCH /api/v1/leads/{id}`	List/create/update leads	staff (scoped)
Leads	`POST /api/v1/leads/{id}/assign`	Assign to counselor/branch	admin
Leads	`POST /api/v1/leads/{id}/convert`	Convert to student + application(s)	counselor
Students	`GET /api/v1/students/{id}/timeline`	Activity timeline	staff/own
Applications	`POST /api/v1/applications` · `POST /{id}/submit` · `POST /{id}/offer`	Admissions lifecycle	admissions
Documents	`POST /api/v1/documents` (pre-signed) · `GET /{id}/download`	Upload/download (logged)	policy
Finance	`POST /api/v1/invoices` · `POST /invoices/{id}/payments`	Invoicing & receipts	finance
Commission	`POST /api/v1/commissions/{id}/transition`	Advance lifecycle state	finance
Agents	`GET /api/v1/agent/students` · `GET /agent/commissions`	Agent portal (own only)	agent
Visa	`POST /api/v1/visa-cases` · `POST /{id}/transition`	Visa workflow	visa
Analytics	`GET /api/v1/dashboards/executive`	Role-scoped KPIs	management
Notifications	`GET /api/v1/notifications` · `POST /{id}/read`	In-app alerts	any

7.3 Example — lead ingestion

```
POST /api/v1/ingest/leads
Headers: X-Api-Key: <source-key>, Idempotency-Key: <uuid>
Body:
{
  "first_name": "Ali", "last_name": "Khan",
  "email": "ali@example.com", "phone": "+92300...",
  "country": "PK", "intake_route": "info",
  "website_domain": "study-xyz-university.com",
  "campaign_utm": "spring-2026-google",
  "message": "Interested in Computer Science, Bachelor"
}
Response 201: { "lead_id": 10234, "student_id": null, "duplicate": false }
Response 409: { "code":"DUPLICATE","message":"Matched existing student","student_id":552 }
```

---

8. Security & RBAC

8.1 Record-scope model

Every permission is evaluated with a **scope** resolved per request:

Scope	Meaning
`global`	All records (Super Admin, Management, Finance for finance data)
`branch`	Records within the user's branch
`department`	Records within the user's department
`assigned`	Records assigned to the user
`own`	Records the user created/owns (agents, students)

8.2 Module × role access (from proposal, authoritative baseline)

Module	Admin	Counselor	Admissions	Finance	Visa	Agent	Student
Dashboard	Full	Assigned	Full	Full	Full	Own	Own
Leads	Full	Assigned	View	—	—	Create/Upload	—
Student Profile	Full	Assigned	Full	View	View	Own	Own
Applications	Full	View	Full	View	View	Own	Own
Universities	Full	View	Full	—	—	View	View
Scholarships	Full	View	Full	—	—	View	View
Documents	Full	Assigned	Full	View	Full	Create/Upload	Create/Upload
Finance	Full	—	View	Full	—	View	Own
Commission	Full	View(own)	View	Full	—	Own	—
Visa	Full	View	View	View	Full	View	Own
Reports	Full	Own	Own	Own	Own	Own	—
User Management	Full	—	—	—	—	—	—

(Full RBAC CRUD matrix with per-action granularity is delivered in Phase 7.)

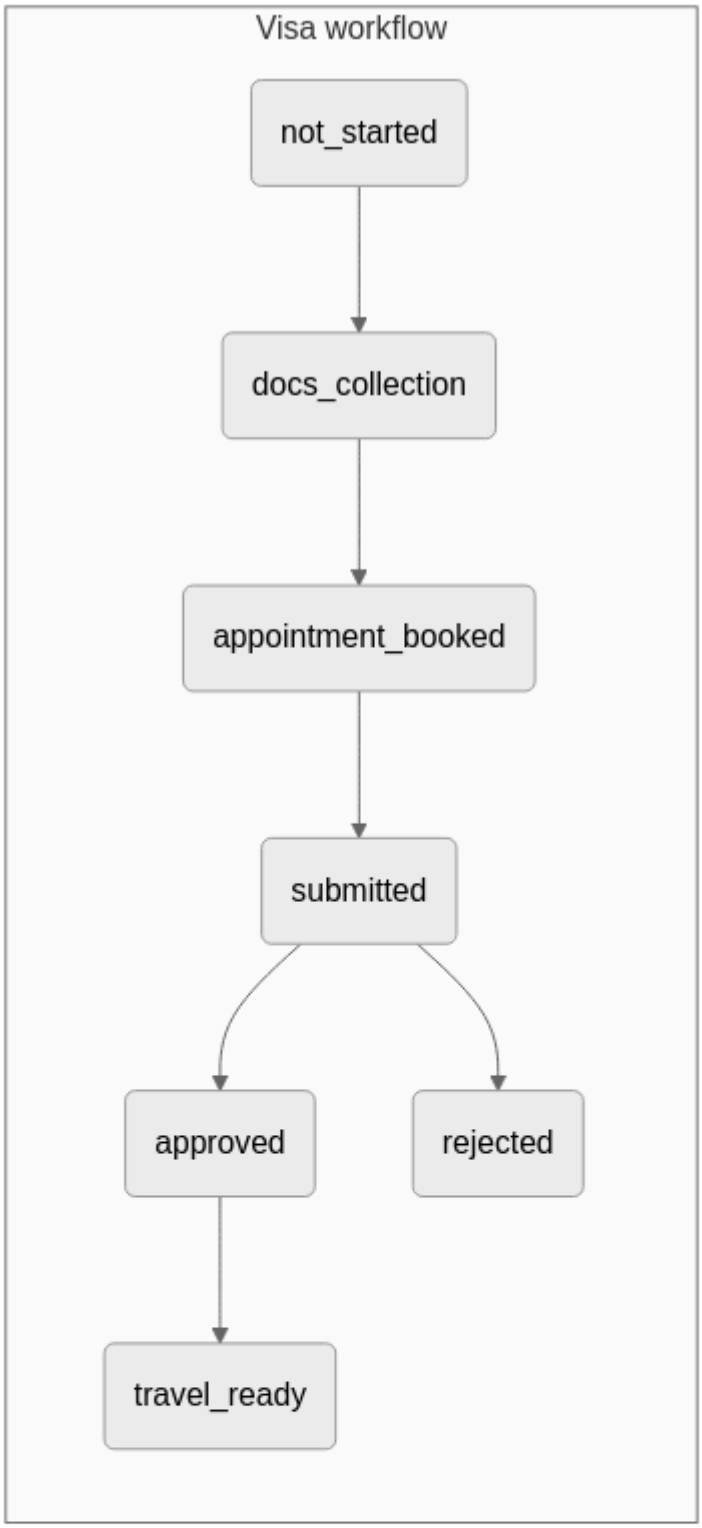
8.3 Controls

- Deny-by-default; policies on every endpoint; 2FA option for staff/finance.
- Encryption in transit (TLS 1.2+) and at rest (DB + object storage).
- Per-source API keys, rate limiting, WAF, input validation, output encoding, CSRF for cookie flows, signed pre-signed URLs for documents.
- Immutable audit + document access logs; password hashing (bcrypt/argon2); secrets in a vault/ENV, never in code.

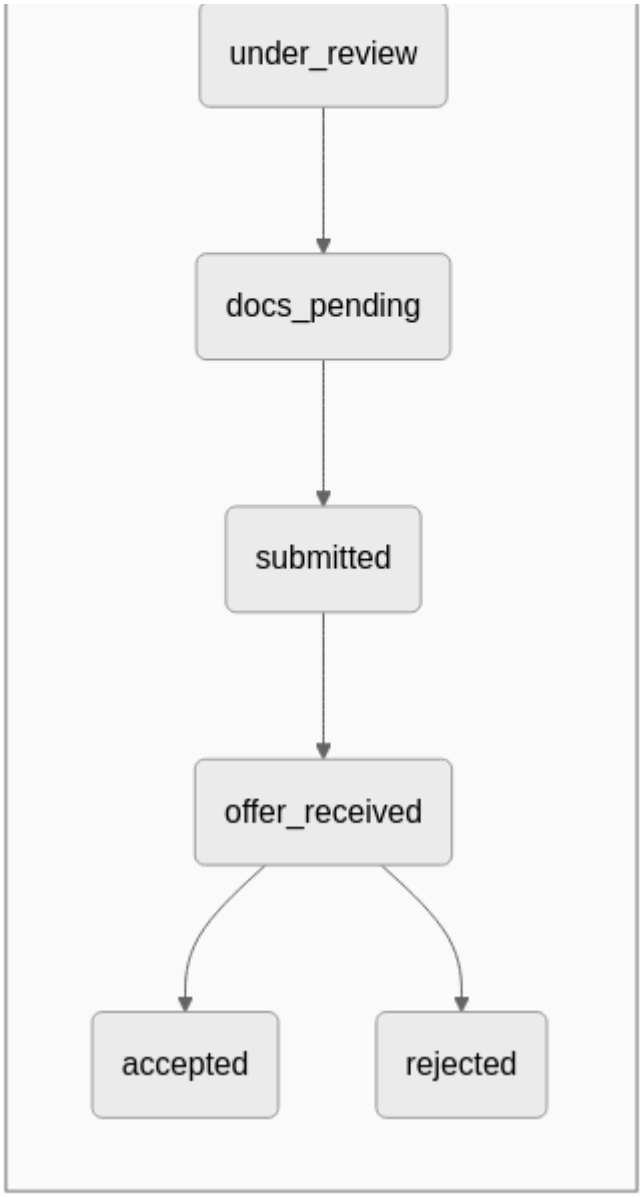
---

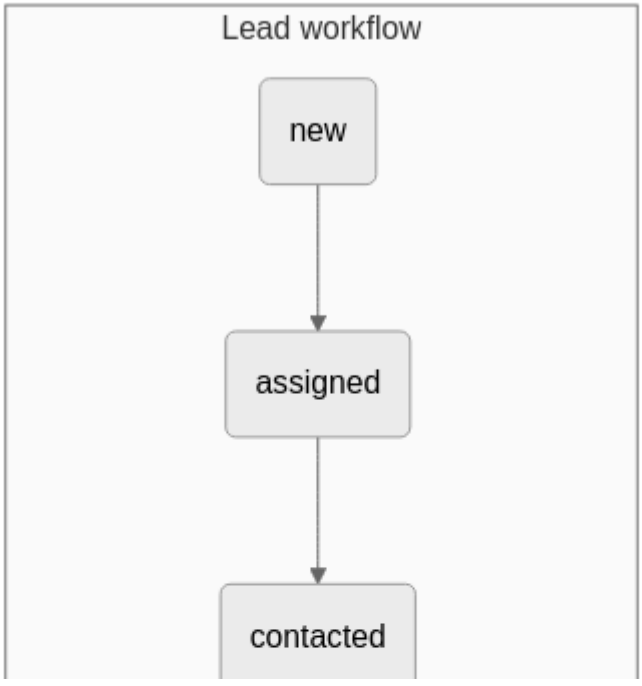
9. Workflow Engine (technical)

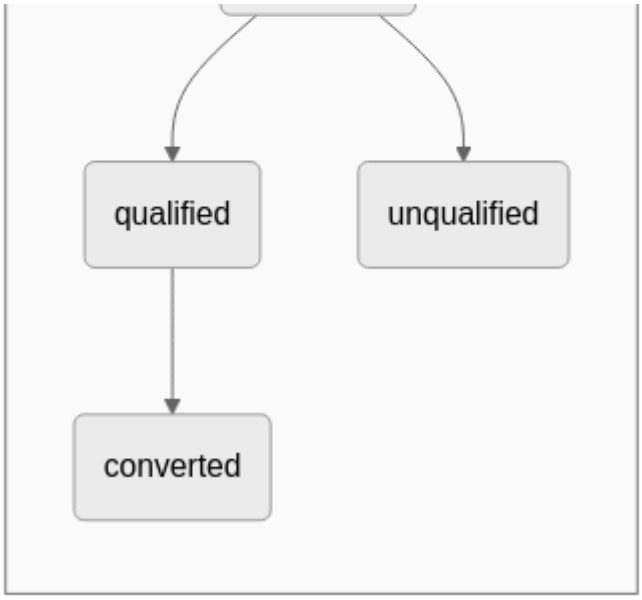
State machines are **data-driven** (`workflow_definitions`) so operations can change states, transitions, SLAs and escalation ladders without redeploying.


Visa workflow
not_started
docs_collection
appointment_booked
submitted
approved
rejected
travel_ready


Application workflow
draft


under_review
docs_pending
submitted
offer_received
accepted
rejected

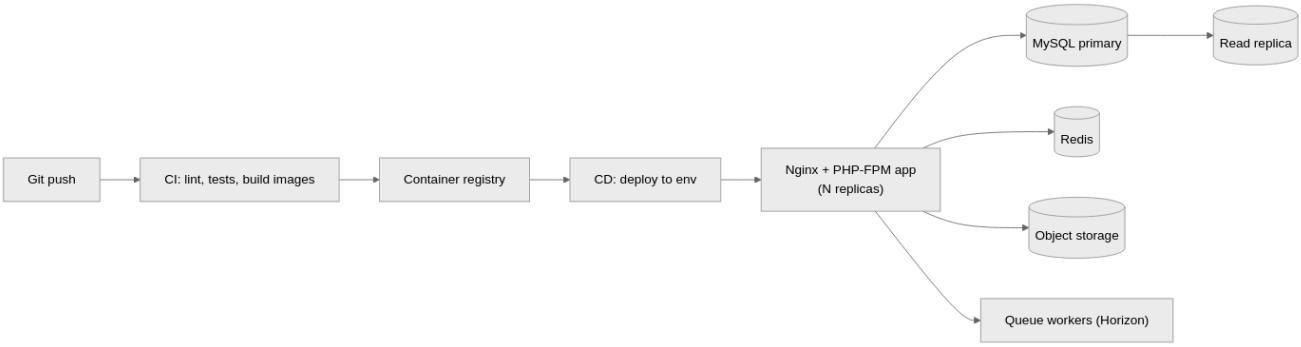

Lead workflow
new
assigned
contacted



- **Transitions** enforce guards (e.g., cannot `submit` an application until checklist complete), fire **side-effects** (create invoice on offer acceptance, open visa case on admission), write **history**, and generate the **next task** with an SLA due date.
- **Escalation:** scheduler scans for SLA breaches → notifies owner → manager → department head per ladder in `settings`.
- **Automation rules** (`automation_rules`) allow event → condition → action (notify, assign, create task, change status) configuration.

---

10. Deployment & DevOps



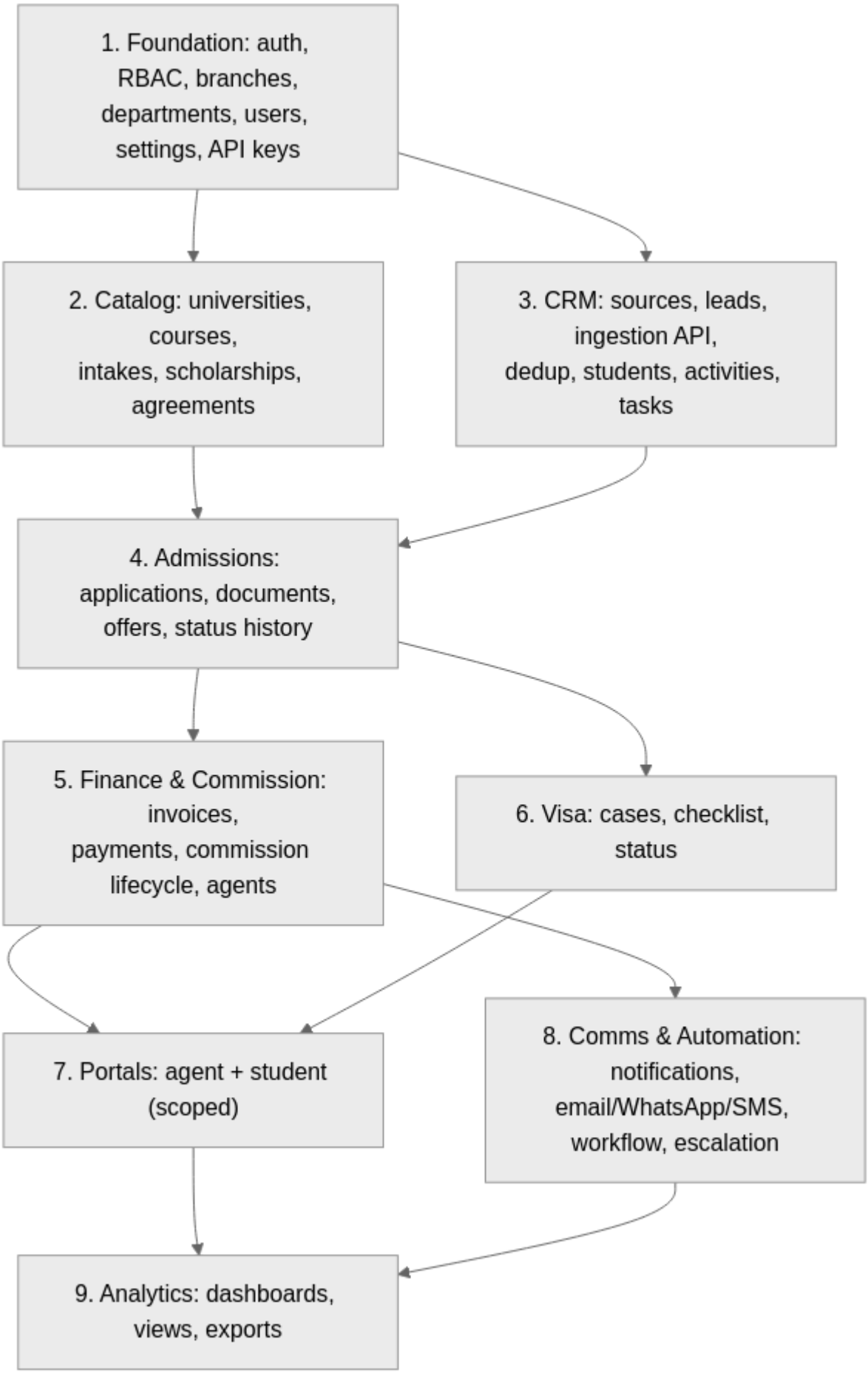
- **Containers:** app (Nginx+PHP-FPM), queue workers, scheduler, MySQL, Redis, MinIO/S3, reverse proxy.
- **CI/CD:** GitHub Actions → test → build → push → deploy (rolling/blue-green). DB migrations run in a controlled step.
- **Scaling:** stateless API scales horizontally; MySQL read replica for reports; Redis for cache/queues; workers scale by queue depth.
- **Backups/DR:** daily full + binlog PITR; tested restores; RPO ≤ 1h, RTO ≤ 4h; documents versioned in object storage.

- **Monitoring:** Sentry (errors), Prometheus/Grafana (metrics), Loki/ELK (logs), uptime checks, alerting on queue depth, job failures, slow queries, disk.

11. Non-Functional Requirements (technical targets)

Area	Requirement
Performance	API p95 < 400 ms (standard reads); dashboards cached 5–15 min
Scalability	Thousands of concurrent users; horizontal API scaling; read replica
Availability	99.5%+ production
Security	Least privilege, TLS, encryption at rest, audit logging, 2FA option
Data integrity	FKs, transactions, soft deletes, immutable financial history
Recoverability	Daily backups + PITR; RPO ≤ 1h, RTO ≤ 4h
Observability	Centralized structured logs, metrics, error tracking, health endpoints
Compliance	Access logging for sensitive documents; configurable retention
Maintainability	Layered architecture, migrations, seeders, automated tests, coding standards
Localization	utf8mb4, multi-currency, timezone-aware timestamps (UTC storage)

12. Build Sequence (technical dependencies)


1. Foundation: auth, RBAC, branches, departments, users, settings, API keys
2. Catalog: universities, courses, intakes, scholarships, agreements
3. CRM: sources, leads, ingestion API, dedup, students, activities, tasks
4. Admissions: applications, documents, offers, status history
5. Finance & Commission: invoices, payments, commission lifecycle, agents
6. Visa: cases, checklist, status
7. Portals: agent + student (scoped)
8. Comms & Automation: notifications, email/WhatsApp/SMS, workflow, escalation
9. Analytics: dashboards, views, exports

Detailed sprint-by-sprint roadmap with estimates is delivered in Phase 11.

Appendix A — Enumerations (reference)

Entity	Statuses
Lead	new, assigned, contacted, qualified, converted, unqualified, lost, duplicate
Student	active, in_process, enrolled, arrived, withdrawn, inactive
Application	draft, under_review, documents_pending, submitted, offer_received, accepted, rejected, withdrawn, deferred
Invoice	draft, issued, partially_paid, paid, cancelled, overdue
Commission	expected, pending_arrival, eligible, invoiced, received
Agent payable	payable, paid
Visa	not_started, documents_collection, appointment_booked, submitted, approved, rejected, travel_ready
Document	pending, verified, rejected
Task	open, in_progress, done, overdue, cancelled

Appendix B — Companion deliverable

`schema.sql` — executable MySQL 8 DDL for the tables above (foundation, CRM, catalog, admissions, documents, finance/commission, agents, visa, workflow, audit) with keys, indexes and constraints.

Appendix C — MySQL Schema (`schema.sql`)

The complete executable DDL is reproduced below and provided as a separate `schema.sql` file.

```
-- ========================================================================
-- CAS Education ERP & CRM Platform  |  MySQL 8 Schema (schema.sql)
-- Engine: InnoDB  |  Charset: utf8mb4  |  Companion to Technical Report
-- Conventions: BIGINT UNSIGNED PKs; soft delete (deleted_at);
--   audit columns created_by/updated_by; money DECIMAL(15,2)+currency.
-- NOTE: created_by/updated_by FKs to users are added where sensible but
--   left un-constrained on high-churn tables to ease seeding; enable in prod.
-- ========================================================================
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- ============================ FOUNDATION ================================
CREATE TABLE countries (
  id           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(100) NOT NULL,
  iso2         CHAR(2) NOT NULL,
  phone_code   VARCHAR(8) NULL,
  created_at   TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_countries_iso2 (iso2)
```

```
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE branches (
  id           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(150) NOT NULL,
  code         VARCHAR(30) NOT NULL,
  country_id   BIGINT UNSIGNED NULL,
  city         VARCHAR(100) NULL,
  is_active    TINYINT(1) NOT NULL DEFAULT 1,
  created_at   TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_branches_code (code),
  KEY idx_branches_country (country_id),
  CONSTRAINT fk_branches_country FOREIGN KEY (country_id) REFERENCES countries(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE departments (
  id           BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name         VARCHAR(100) NOT NULL,
  code         VARCHAR(30) NOT NULL,
  is_active    TINYINT(1) NOT NULL DEFAULT 1,
  created_at   TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_departments_code (code)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE users (
  id             BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name           VARCHAR(150) NOT NULL,
  email          VARCHAR(190) NOT NULL,
  phone          VARCHAR(30) NULL,
  password       VARCHAR(255) NOT NULL,
  branch_id      BIGINT UNSIGNED NULL,
  department_id  BIGINT UNSIGNED NULL,
  status         VARCHAR(20) NOT NULL DEFAULT 'active',
  two_fa_secret  VARCHAR(255) NULL,
  last_login_at  TIMESTAMP NULL,
  created_at     TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_users_email (email),
  KEY idx_users_branch (branch_id), KEY idx_users_dept (department_id),
  CONSTRAINT fk_users_branch FOREIGN KEY (branch_id) REFERENCES branches(id),
  CONSTRAINT fk_users_dept FOREIGN KEY (department_id) REFERENCES departments(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- RBAC (spatie-compatible)
CREATE TABLE roles (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL, guard_name VARCHAR(50) NOT NULL DEFAULT 'web',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_roles_name_guard (name, guard_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE permissions (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, guard_name VARCHAR(50) NOT NULL DEFAULT 'web',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_permissions_name_guard (name, guard_name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE role_permissions (
  permission_id BIGINT UNSIGNED NOT NULL,
  role_id       BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY (permission_id, role_id),
  CONSTRAINT fk_rp_perm FOREIGN KEY (permission_id) REFERENCES permissions(id) ON DELETE CASCADE,
  CONSTRAINT fk_rp_role FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE model_has_roles (
  role_id    BIGINT UNSIGNED NOT NULL,
  model_type VARCHAR(150) NOT NULL,
  model_id   BIGINT UNSIGNED NOT NULL,
  PRIMARY KEY (role_id, model_id, model_type),
  KEY idx_mhr_model (model_id, model_type),
```

```
  CONSTRAINT fk_mhr_role FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE settings (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  `key` VARCHAR(150) NOT NULL, `value` JSON NULL, `group` VARCHAR(80) NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_settings_key (`key`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE api_sources (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  type VARCHAR(30) NOT NULL DEFAULT 'website',   -- website|agent|campaign|manual
  website_url VARCHAR(255) NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE api_keys (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  api_source_id BIGINT UNSIGNED NOT NULL,
  key_hash VARCHAR(255) NOT NULL,
  scopes JSON NULL,
  rate_limit INT UNSIGNED NOT NULL DEFAULT 60,
  revoked_at TIMESTAMP NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_api_keys_hash (key_hash),
  KEY idx_api_keys_source (api_source_id),
  CONSTRAINT fk_apikeys_source FOREIGN KEY (api_source_id) REFERENCES api_sources(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ============================== CRM ==================================
CREATE TABLE marketing_sources (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL, channel VARCHAR(60) NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE campaigns (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL,
  marketing_source_id BIGINT UNSIGNED NULL,
  utm_source VARCHAR(120) NULL, utm_medium VARCHAR(120) NULL, utm_campaign VARCHAR(120) NULL,
  start_date DATE NULL, end_date DATE NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  KEY idx_campaigns_ms (marketing_source_id),
  CONSTRAINT fk_campaigns_ms FOREIGN KEY (marketing_source_id) REFERENCES marketing_sources(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE website_sources (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  api_source_id BIGINT UNSIGNED NULL,
  domain VARCHAR(190) NOT NULL,
  university_focus VARCHAR(190) NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_website_domain (domain),
  KEY idx_website_source (api_source_id),
  CONSTRAINT fk_website_source FOREIGN KEY (api_source_id) REFERENCES api_sources(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE agents (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  agent_code VARCHAR(40) NOT NULL,
  name VARCHAR(180) NOT NULL,
  type VARCHAR(20) NOT NULL DEFAULT 'partner',   -- partner|sub|sub_sub
  parent_agent_id BIGINT UNSIGNED NULL,
  email VARCHAR(190) NULL, phone VARCHAR(30) NULL,
  country_id BIGINT UNSIGNED NULL,
```

```
  status VARCHAR(20) NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_agents_code (agent_code),
  KEY idx_agents_parent (parent_agent_id),
  CONSTRAINT fk_agents_parent FOREIGN KEY (parent_agent_id) REFERENCES agents(id),
  CONSTRAINT fk_agents_country FOREIGN KEY (country_id) REFERENCES countries(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE students (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  student_code VARCHAR(40) NOT NULL,
  first_name VARCHAR(100) NOT NULL, last_name VARCHAR(100) NOT NULL,
  email VARCHAR(190) NULL, phone VARCHAR(30) NULL,
  passport_no VARCHAR(50) NULL, dob DATE NULL, gender VARCHAR(15) NULL,
  country_id BIGINT UNSIGNED NULL, branch_id BIGINT UNSIGNED NULL,
  agent_id BIGINT UNSIGNED NULL, assigned_counselor_id BIGINT UNSIGNED NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'active',
  created_by BIGINT UNSIGNED NULL, updated_by BIGINT UNSIGNED NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_students_code (student_code),
  KEY idx_students_passport (passport_no),
  KEY idx_students_email (email), KEY idx_students_phone (phone),
  KEY idx_students_counselor (assigned_counselor_id),
  KEY idx_students_agent (agent_id), KEY idx_students_status (status),
  CONSTRAINT fk_students_country FOREIGN KEY (country_id) REFERENCES countries(id),
  CONSTRAINT fk_students_branch FOREIGN KEY (branch_id) REFERENCES branches(id),
  CONSTRAINT fk_students_agent FOREIGN KEY (agent_id) REFERENCES agents(id),
  CONSTRAINT fk_students_counselor FOREIGN KEY (assigned_counselor_id) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE leads (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  first_name VARCHAR(100) NULL, last_name VARCHAR(100) NULL,
  email VARCHAR(190) NULL, phone VARCHAR(30) NULL, passport_no VARCHAR(50) NULL,
  country_id BIGINT UNSIGNED NULL,
  website_source_id BIGINT UNSIGNED NULL, marketing_source_id BIGINT UNSIGNED NULL,
  campaign_id BIGINT UNSIGNED NULL, agent_id BIGINT UNSIGNED NULL,
  assigned_user_id BIGINT UNSIGNED NULL,
  intake_route VARCHAR(10) NOT NULL DEFAULT 'info',  -- info|full
  status VARCHAR(20) NOT NULL DEFAULT 'new',
  student_id BIGINT UNSIGNED NULL, duplicate_of BIGINT UNSIGNED NULL,
  message TEXT NULL, meta JSON NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  KEY idx_leads_status_assignee_date (status, assigned_user_id, created_at),
  KEY idx_leads_email (email), KEY idx_leads_phone (phone), KEY idx_leads_passport (passport_no),
  KEY idx_leads_website (website_source_id), KEY idx_leads_campaign (campaign_id),
  CONSTRAINT fk_leads_country FOREIGN KEY (country_id) REFERENCES countries(id),
  CONSTRAINT fk_leads_website FOREIGN KEY (website_source_id) REFERENCES website_sources(id),
  CONSTRAINT fk_leads_ms FOREIGN KEY (marketing_source_id) REFERENCES marketing_sources(id),
  CONSTRAINT fk_leads_campaign FOREIGN KEY (campaign_id) REFERENCES campaigns(id),
  CONSTRAINT fk_leads_agent FOREIGN KEY (agent_id) REFERENCES agents(id),
  CONSTRAINT fk_leads_assignee FOREIGN KEY (assigned_user_id) REFERENCES users(id),
  CONSTRAINT fk_leads_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_leads_dupof FOREIGN KEY (duplicate_of) REFERENCES leads(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE student_guardians (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  student_id BIGINT UNSIGNED NOT NULL,
  name VARCHAR(150) NOT NULL, relation VARCHAR(50) NULL,
  phone VARCHAR(30) NULL, email VARCHAR(190) NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_guardians_student (student_id),
  CONSTRAINT fk_guardians_student FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE activities (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  student_id BIGINT UNSIGNED NOT NULL, user_id BIGINT UNSIGNED NULL,
  type VARCHAR(30) NOT NULL,           -- note|call|email|status|system
  body TEXT NULL, meta JSON NULL,
```

```
  created_at TIMESTAMP NULL,
  KEY idx_activities_student (student_id, created_at),
  CONSTRAINT fk_activities_student FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,
  CONSTRAINT fk_activities_user FOREIGN KEY (user_id) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ============================ CATALOG ================================
CREATE TABLE universities (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(190) NOT NULL, code VARCHAR(40) NOT NULL,
  country_id BIGINT UNSIGNED NULL, city VARCHAR(100) NULL, website VARCHAR(190) NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_universities_code (code),
  CONSTRAINT fk_universities_country FOREIGN KEY (country_id) REFERENCES countries(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE university_contacts (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  university_id BIGINT UNSIGNED NOT NULL,
  name VARCHAR(150) NOT NULL, email VARCHAR(190) NULL, phone VARCHAR(30) NULL, role VARCHAR(80) NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_unicontacts_uni (university_id),
  CONSTRAINT fk_unicontacts_uni FOREIGN KEY (university_id) REFERENCES universities(id) ON DELETE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE courses (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  university_id BIGINT UNSIGNED NOT NULL,
  name VARCHAR(190) NOT NULL, level VARCHAR(20) NOT NULL DEFAULT 'bachelor',
  duration_months SMALLINT UNSIGNED NULL,
  tuition_fee DECIMAL(15,2) NULL, currency CHAR(3) NULL,
  requirements JSON NULL, status VARCHAR(20) NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  KEY idx_courses_uni (university_id),
  CONSTRAINT fk_courses_uni FOREIGN KEY (university_id) REFERENCES universities(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE intakes (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  university_id BIGINT UNSIGNED NOT NULL, course_id BIGINT UNSIGNED NULL,
  name VARCHAR(80) NOT NULL, start_date DATE NULL, application_deadline DATE NULL,
  seats INT UNSIGNED NULL, status VARCHAR(20) NOT NULL DEFAULT 'open',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_intakes_uni (university_id), KEY idx_intakes_course (course_id),
  CONSTRAINT fk_intakes_uni FOREIGN KEY (university_id) REFERENCES universities(id),
  CONSTRAINT fk_intakes_course FOREIGN KEY (course_id) REFERENCES courses(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE scholarships (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  university_id BIGINT UNSIGNED NOT NULL,
  name VARCHAR(190) NOT NULL, type VARCHAR(60) NULL, coverage VARCHAR(120) NULL,
  criteria JSON NULL, deadline DATE NULL, status VARCHAR(20) NOT NULL DEFAULT 'active',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  KEY idx_scholarships_uni (university_id),
  CONSTRAINT fk_scholarships_uni FOREIGN KEY (university_id) REFERENCES universities(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE university_agreements (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  university_id BIGINT UNSIGNED NOT NULL, course_id BIGINT UNSIGNED NULL,
  commission_type VARCHAR(20) NOT NULL DEFAULT 'percent', -- fixed|percent|tiered
  commission_value DECIMAL(15,2) NOT NULL DEFAULT 0,
  currency CHAR(3) NULL,
  eligibility_trigger VARCHAR(60) NOT NULL DEFAULT 'arrival_first_year_paid',
  valid_from DATE NULL, valid_to DATE NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_agreements_uni (university_id), KEY idx_agreements_course (course_id),
  CONSTRAINT fk_agreements_uni FOREIGN KEY (university_id) REFERENCES universities(id),
```

```
  CONSTRAINT fk_agreements_course FOREIGN KEY (course_id) REFERENCES courses(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE agent_agreements (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  agent_id BIGINT UNSIGNED NOT NULL,
  share_type VARCHAR(20) NOT NULL DEFAULT 'percent',  -- percent|fixed
  share_value DECIMAL(15,2) NOT NULL DEFAULT 0,
  applies_to JSON NULL, valid_from DATE NULL, valid_to DATE NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_agentagreements_agent (agent_id),
  CONSTRAINT fk_agentagreements_agent FOREIGN KEY (agent_id) REFERENCES agents(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ======================== ADMISSIONS/DOCS ========================
CREATE TABLE applications (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  application_code VARCHAR(40) NOT NULL,
  student_id BIGINT UNSIGNED NOT NULL,
  university_id BIGINT UNSIGNED NOT NULL, course_id BIGINT UNSIGNED NOT NULL,
  intake_id BIGINT UNSIGNED NULL, scholarship_id BIGINT UNSIGNED NULL,
  agent_id BIGINT UNSIGNED NULL, assigned_admissions_id BIGINT UNSIGNED NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'draft',
  submitted_at TIMESTAMP NULL,
  created_by BIGINT UNSIGNED NULL, updated_by BIGINT UNSIGNED NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_applications_code (application_code),
  KEY idx_apps_student (student_id),
  KEY idx_apps_status_admissions (status, assigned_admissions_id),
  KEY idx_apps_uni (university_id), KEY idx_apps_course (course_id),
  CONSTRAINT fk_apps_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_apps_uni FOREIGN KEY (university_id) REFERENCES universities(id),
  CONSTRAINT fk_apps_course FOREIGN KEY (course_id) REFERENCES courses(id),
  CONSTRAINT fk_apps_intake FOREIGN KEY (intake_id) REFERENCES intakes(id),
  CONSTRAINT fk_apps_scholarship FOREIGN KEY (scholarship_id) REFERENCES scholarships(id),
  CONSTRAINT fk_apps_agent FOREIGN KEY (agent_id) REFERENCES agents(id),
  CONSTRAINT fk_apps_admissions FOREIGN KEY (assigned_admissions_id) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE application_status_history (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  application_id BIGINT UNSIGNED NOT NULL,
  from_status VARCHAR(30) NULL, to_status VARCHAR(30) NOT NULL,
  changed_by BIGINT UNSIGNED NULL, note VARCHAR(500) NULL,
  created_at TIMESTAMP NULL,
  KEY idx_ash_app (application_id, created_at),
  CONSTRAINT fk_ash_app FOREIGN KEY (application_id) REFERENCES applications(id) ON DELETE CASCADE,
  CONSTRAINT fk_ash_user FOREIGN KEY (changed_by) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE document_types (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL, category VARCHAR(60) NULL,
  is_sensitive TINYINT(1) NOT NULL DEFAULT 0, required_for JSON NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_doctypes_name (name)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE documents (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  document_type_id BIGINT UNSIGNED NULL,
  documentable_type VARCHAR(150) NOT NULL, documentable_id BIGINT UNSIGNED NOT NULL,
  file_path VARCHAR(500) NOT NULL, file_name VARCHAR(255) NOT NULL,
  mime VARCHAR(120) NULL, size BIGINT UNSIGNED NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  version INT UNSIGNED NOT NULL DEFAULT 1,
  verified_by BIGINT UNSIGNED NULL,
  created_by BIGINT UNSIGNED NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  KEY idx_documents_poly (documentable_type, documentable_id),
  KEY idx_documents_type (document_type_id),
```

```
  CONSTRAINT fk_documents_type FOREIGN KEY (document_type_id) REFERENCES document_types(id),
  CONSTRAINT fk_documents_verifier FOREIGN KEY (verified_by) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE document_access_logs (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  document_id BIGINT UNSIGNED NOT NULL, user_id BIGINT UNSIGNED NULL,
  action VARCHAR(20) NOT NULL, ip VARCHAR(45) NULL, created_at TIMESTAMP NULL,
  KEY idx_dal_doc (document_id, created_at),
  CONSTRAINT fk_dal_doc FOREIGN KEY (document_id) REFERENCES documents(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE offers (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  application_id BIGINT UNSIGNED NOT NULL,
  offer_type VARCHAR(20) NOT NULL DEFAULT 'conditional',
  reference_no VARCHAR(80) NULL, issued_date DATE NULL, expiry_date DATE NULL,
  document_id BIGINT UNSIGNED NULL, status VARCHAR(20) NOT NULL DEFAULT 'issued',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_offers_app (application_id),
  CONSTRAINT fk_offers_app FOREIGN KEY (application_id) REFERENCES applications(id) ON DELETE CASCADE,
  CONSTRAINT fk_offers_doc FOREIGN KEY (document_id) REFERENCES documents(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ====================== FINANCE & COMMISSION ======================
CREATE TABLE invoices (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  invoice_no VARCHAR(50) NOT NULL,
  party_type VARCHAR(20) NOT NULL,      -- student|agent|university
  party_id BIGINT UNSIGNED NOT NULL,
  application_id BIGINT UNSIGNED NULL,
  currency CHAR(3) NOT NULL DEFAULT 'USD',
  subtotal DECIMAL(15,2) NOT NULL DEFAULT 0, tax DECIMAL(15,2) NOT NULL DEFAULT 0,
  total DECIMAL(15,2) NOT NULL DEFAULT 0, amount_paid DECIMAL(15,2) NOT NULL DEFAULT 0,
  status VARCHAR(20) NOT NULL DEFAULT 'draft',
  issued_at TIMESTAMP NULL, due_at TIMESTAMP NULL,
  created_by BIGINT UNSIGNED NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_invoices_no (invoice_no),
  KEY idx_invoices_party (party_type, party_id),
  KEY idx_invoices_status (status), KEY idx_invoices_app (application_id),
  CONSTRAINT fk_invoices_app FOREIGN KEY (application_id) REFERENCES applications(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE invoice_items (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  invoice_id BIGINT UNSIGNED NOT NULL,
  description VARCHAR(255) NOT NULL, qty DECIMAL(10,2) NOT NULL DEFAULT 1,
  unit_price DECIMAL(15,2) NOT NULL DEFAULT 0, amount DECIMAL(15,2) NOT NULL DEFAULT 0,
  KEY idx_invitems_inv (invoice_id),
  CONSTRAINT fk_invitems_inv FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE payments (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  payment_no VARCHAR(50) NOT NULL, invoice_id BIGINT UNSIGNED NOT NULL,
  amount DECIMAL(15,2) NOT NULL, currency CHAR(3) NOT NULL DEFAULT 'USD',
  method VARCHAR(40) NULL, reference VARCHAR(120) NULL,
  document_id BIGINT UNSIGNED NULL, received_at TIMESTAMP NULL,
  recorded_by BIGINT UNSIGNED NULL, created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  UNIQUE KEY uq_payments_no (payment_no),
  KEY idx_payments_inv (invoice_id),
  CONSTRAINT fk_payments_inv FOREIGN KEY (invoice_id) REFERENCES invoices(id),
  CONSTRAINT fk_payments_doc FOREIGN KEY (document_id) REFERENCES documents(id),
  CONSTRAINT fk_payments_user FOREIGN KEY (recorded_by) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE commissions (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  application_id BIGINT UNSIGNED NOT NULL,
  university_id BIGINT UNSIGNED NOT NULL, agreement_id BIGINT UNSIGNED NULL,
```

```
  currency CHAR(3) NOT NULL DEFAULT 'USD',
  expected_amount DECIMAL(15,2) NOT NULL DEFAULT 0,
  eligible_amount DECIMAL(15,2) NOT NULL DEFAULT 0,
  invoiced_amount DECIMAL(15,2) NOT NULL DEFAULT 0,
  received_amount DECIMAL(15,2) NOT NULL DEFAULT 0,
  status VARCHAR(20) NOT NULL DEFAULT 'expected',
  eligibility_confirmed_at TIMESTAMP NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  UNIQUE KEY uq_commissions_app (application_id),
  KEY idx_commissions_status_uni (status, university_id),
  CONSTRAINT fk_commissions_app FOREIGN KEY (application_id) REFERENCES applications(id),
  CONSTRAINT fk_commissions_uni FOREIGN KEY (university_id) REFERENCES universities(id),
  CONSTRAINT fk_commissions_agreement FOREIGN KEY (agreement_id) REFERENCES university_agreements(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE commission_status_history (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  commission_id BIGINT UNSIGNED NOT NULL,
  from_status VARCHAR(20) NULL, to_status VARCHAR(20) NOT NULL,
  amount DECIMAL(15,2) NULL, reference VARCHAR(120) NULL,
  changed_by BIGINT UNSIGNED NULL, created_at TIMESTAMP NULL,
  KEY idx_csh_commission (commission_id, created_at),
  CONSTRAINT fk_csh_commission FOREIGN KEY (commission_id) REFERENCES commissions(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE agent_payables (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  commission_id BIGINT UNSIGNED NOT NULL, agent_id BIGINT UNSIGNED NOT NULL,
  amount DECIMAL(15,2) NOT NULL DEFAULT 0, currency CHAR(3) NOT NULL DEFAULT 'USD',
  status VARCHAR(20) NOT NULL DEFAULT 'payable',   -- payable|paid
  computed_at TIMESTAMP NULL, created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_payables_commission (commission_id), KEY idx_payables_agent (agent_id),
  CONSTRAINT fk_payables_commission FOREIGN KEY (commission_id) REFERENCES commissions(id),
  CONSTRAINT fk_payables_agent FOREIGN KEY (agent_id) REFERENCES agents(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE agent_payments (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  agent_payable_id BIGINT UNSIGNED NOT NULL,
  amount DECIMAL(15,2) NOT NULL, method VARCHAR(40) NULL, reference VARCHAR(120) NULL,
  document_id BIGINT UNSIGNED NULL, paid_at TIMESTAMP NULL,
  recorded_by BIGINT UNSIGNED NULL, created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_agentpay_payable (agent_payable_id),
  CONSTRAINT fk_agentpay_payable FOREIGN KEY (agent_payable_id) REFERENCES agent_payables(id),
  CONSTRAINT fk_agentpay_doc FOREIGN KEY (document_id) REFERENCES documents(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ============================== VISA ================================
CREATE TABLE visa_cases (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  application_id BIGINT UNSIGNED NOT NULL, student_id BIGINT UNSIGNED NOT NULL,
  assigned_visa_officer_id BIGINT UNSIGNED NULL,
  status VARCHAR(30) NOT NULL DEFAULT 'not_started',
  appointment_at TIMESTAMP NULL, submitted_at TIMESTAMP NULL,
  decision_at TIMESTAMP NULL, travel_ready_at TIMESTAMP NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  KEY idx_visa_app (application_id), KEY idx_visa_student (student_id),
  KEY idx_visa_status (status),
  CONSTRAINT fk_visa_app FOREIGN KEY (application_id) REFERENCES applications(id),
  CONSTRAINT fk_visa_student FOREIGN KEY (student_id) REFERENCES students(id),
  CONSTRAINT fk_visa_officer FOREIGN KEY (assigned_visa_officer_id) REFERENCES users(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE visa_checklist_items (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  visa_case_id BIGINT UNSIGNED NOT NULL, name VARCHAR(150) NOT NULL,
  is_required TINYINT(1) NOT NULL DEFAULT 1, document_id BIGINT UNSIGNED NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'pending',
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_visaitems_case (visa_case_id),
  CONSTRAINT fk_visaitems_case FOREIGN KEY (visa_case_id) REFERENCES visa_cases(id) ON DELETE CASCADE,
```

```
  CONSTRAINT fk_visaitems_doc FOREIGN KEY (document_id) REFERENCES documents(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE visa_status_history (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  visa_case_id BIGINT UNSIGNED NOT NULL,
  from_status VARCHAR(30) NULL, to_status VARCHAR(30) NOT NULL,
  changed_by BIGINT UNSIGNED NULL, note VARCHAR(500) NULL, created_at TIMESTAMP NULL,
  KEY idx_vsh_case (visa_case_id, created_at),
  CONSTRAINT fk_vsh_case FOREIGN KEY (visa_case_id) REFERENCES visa_cases(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- ================= WORKFLOW / COMMS / SYSTEM ======================
CREATE TABLE tasks (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(190) NOT NULL, description TEXT NULL,
  assignee_id BIGINT UNSIGNED NULL, student_id BIGINT UNSIGNED NULL,
  related_type VARCHAR(150) NULL, related_id BIGINT UNSIGNED NULL,
  due_at TIMESTAMP NULL, priority VARCHAR(10) NOT NULL DEFAULT 'normal',
  status VARCHAR(20) NOT NULL DEFAULT 'open', escalated_at TIMESTAMP NULL,
  created_by BIGINT UNSIGNED NULL,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL, deleted_at TIMESTAMP NULL,
  KEY idx_tasks_assignee_status_due (assignee_id, status, due_at),
  KEY idx_tasks_student (student_id),
  CONSTRAINT fk_tasks_assignee FOREIGN KEY (assignee_id) REFERENCES users(id),
  CONSTRAINT fk_tasks_student FOREIGN KEY (student_id) REFERENCES students(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE workflow_definitions (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL, entity_type VARCHAR(60) NOT NULL,
  states JSON NULL, transitions JSON NULL, sla JSON NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE workflow_transitions (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  entity_type VARCHAR(60) NOT NULL, entity_id BIGINT UNSIGNED NOT NULL,
  from_state VARCHAR(40) NULL, to_state VARCHAR(40) NOT NULL,
  actor_id BIGINT UNSIGNED NULL, created_at TIMESTAMP NULL,
  KEY idx_wt_entity (entity_type, entity_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE automation_rules (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(150) NOT NULL, trigger_def JSON NULL, conditions JSON NULL, actions JSON NULL,
  is_active TINYINT(1) NOT NULL DEFAULT 1,
  created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE notifications (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NULL, type VARCHAR(80) NOT NULL,
  title VARCHAR(190) NULL, body TEXT NULL, channels JSON NULL,
  read_at TIMESTAMP NULL, created_at TIMESTAMP NULL,
  KEY idx_notifications_user (user_id, read_at),
  CONSTRAINT fk_notifications_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE communications (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  student_id BIGINT UNSIGNED NULL, channel VARCHAR(20) NOT NULL,  -- email|whatsapp|sms
  direction VARCHAR(10) NOT NULL DEFAULT 'out', template VARCHAR(120) NULL,
  body TEXT NULL, status VARCHAR(20) NOT NULL DEFAULT 'queued',
  sent_by BIGINT UNSIGNED NULL, created_at TIMESTAMP NULL, updated_at TIMESTAMP NULL,
  KEY idx_comm_student (student_id, created_at),
  CONSTRAINT fk_comm_student FOREIGN KEY (student_id) REFERENCES students(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE message_logs (
```

```
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  communication_id BIGINT UNSIGNED NOT NULL, provider VARCHAR(60) NULL,
  provider_msg_id VARCHAR(150) NULL, status VARCHAR(30) NULL, error TEXT NULL,
  created_at TIMESTAMP NULL,
  KEY idx_msglogs_comm (communication_id),
  CONSTRAINT fk_msglogs_comm FOREIGN KEY (communication_id) REFERENCES communications(id) ON DELETE
CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE audit_logs (
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT PRIMARY KEY,
  user_id BIGINT UNSIGNED NULL, action VARCHAR(40) NOT NULL,
  auditable_type VARCHAR(150) NOT NULL, auditable_id BIGINT UNSIGNED NOT NULL,
  old_values JSON NULL, new_values JSON NULL,
  ip VARCHAR(45) NULL, user_agent VARCHAR(255) NULL, created_at TIMESTAMP NULL,
  KEY idx_audit_entity (auditable_type, auditable_id, created_at),
  KEY idx_audit_user (user_id, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

SET FOREIGN_KEY_CHECKS = 1;
-- ========================== END OF SCHEMA ==========================
```